Locksmith Lab

Analysis

Nikola Panayotov

Table of Contents

1 Introduction2
2 Project description2
2.1 Problem and intended users2
2.2 Project goals2
3 Scope and constraints2
4 Analysis approach and findings3
4.1 Research methods to use next3
4.2 Scope decisions4
5 Requirement priorities4
6 Must have requirements5
7 Risks and validation priorities6
References6

1 Introduction

This analysis defines the problem, scope and requirements for the Security Playground project. The aim is to build a small networked web application (SecureNotes) inside a controlled environment, attack it with fundamental Linux tools, improve it, and test it again. The main priorities are a safe test setup, evidence that can be repeated, and a scope small enough to leave time for testing and documentation.

The project answers the challenge question: how can a basic networked web application be built within the EDU Cloud, attacked, and improved using fundamental Linux or Windows tools?

2 Project description

SecureNotes is a simple website where users create an account, log in, and create, view, edit and delete personal notes. The application is a means to an end. It contains enough functionality to exercise authentication, authorization, a database, input handling and network exposure, but it is not meant to be a large product.

The project follows a build, attack, fix and retest cycle. The unsecured version is a working application with realistic weaknesses that runs only in an isolated environment. The secured version fixes those weaknesses and is deployed on the Fontys GNS3 Ubuntu VPS. Both versions are tested with the same test cases so the before and after results can be compared.

2.1 Problem and intended users

Web applications become vulnerable through weak authentication, missing authorization checks, unsafe input handling, insecure network configuration and poor logging. Reading about these problems does not show how they behave in practice, and testing them on real systems would be unsafe and unethical. A safe environment is needed where an application can be broken deliberately and then repaired.

The intended users of this analysis and its results are:

- **The client:** who needs to understand how vulnerabilities arise, are detected and are fixed.
- **Reviewers and assessors:** who need documented evidence of the findings, fixes and retest results.
- **SecureNotes users (simulated):** normal users whose accounts and notes the application must protect from other users and from outsiders.

2.2 Project goals

- Demonstrate authentication, password hashing and access control.
- Find and fix SQL injection and cross-site scripting (XSS) in the application code.
- Run and secure a Linux server using Docker (Docker, n.d.), Nginx, UFW (Ubuntu, n.d.), HTTPS and closed ports.
- Use Nmap, Wireshark, tcpdump and curl to gather evidence, and explain the results.
- Document findings, fixes and retest results clearly enough for someone else to repeat them.

3 Scope and constraints

The scope is limited so that the focus stays on cybersecurity and networking rather than on features.

Area	Decision
In scope	SecureNotes with accounts and personal notes; an unsecured version and a secured version; Docker deployment on Ubuntu; network tests (ports, HTTP versus HTTPS traffic); application tests (authentication, authorization, injection, XSS); server hardening; documentation and reporting.

Area	Decision
Out of scope	Note sharing, file uploads and a polished interface; custom exploits or malware; denial-of-service tests; high availability, scaling and compliance audits.
Safety constraint	The unsecured version is never exposed to the public internet. It runs in the EDU Cloud, a local virtual machine or an isolated Docker network.
Ethical constraint	Only systems owned by the client or provided for this project are scanned or attacked. EDU Cloud usage rules are checked before scanning.
Technical constraint	Stack is React and TypeScript, Node.js and TypeScript (Express), PostgreSQL, Docker, Ubuntu and Nginx. Tools are limited to fundamental ones such as Nmap, Wireshark, tcpdump, curl and UFW.
Environment constraint	The secured version is hosted on an existing Ubuntu VPS. Only ports 22, 80 and 443 may be reachable. Port 5432 and the internal API port must not be public.
Time constraint	The application must stay small enough that most of the time goes into testing, fixing and retesting.

4 Analysis approach and findings

Analysis is a structured investigation of the context, goals, stakeholders and constraints. For this project, the project idea, the challenge description and the OWASP documentation (OWASP Foundation, 2021) define the intended behaviour and the vulnerabilities to test. A comparison of framework documentation (Express, n.d.; Fastify, n.d.) supports the technology choices. Testing follows the planning, execution and reporting structure described in NIST guidance (Scarfone et al., 2008). These activities do not show that the application has already been built or has passed any test.

Question	Current finding	Further evidence needed
What must the project achieve?	Show a full cycle: build a networked application, attack it, fix it and retest it, with evidence for each step.	Confirm with the client that the scope and evidence match the agreed requirements.
What is the minimum viable product?	Accounts and personal notes (create, view, edit, delete) in an unsecured version and a secured version, plus documented tests of both.	Demonstrate every planned test on both versions with recorded results.
Which technology fits the project goals?	TypeScript on both sides allows fast development. Fastify is preferred because schema validation is built in (Fastify, n.d.). PostgreSQL gives a realistic SQL injection target.	Build a small working prototype and confirm Fastify against Express.
Which vulnerabilities should be tested?	The ten weaknesses listed in section 4.2, mapped to the OWASP Top 10 (OWASP Foundation, 2021).	Check that each one can be reproduced in the unsecured version and reliably blocked in the secured version.
What creates the greatest risk?	Exposing the unsecured version publicly and running out of time for retesting.	Verify port bindings before every deployment and plan testing early.

4.1 Research methods to use next

A literature review of the OWASP Top 10 (OWASP Foundation, 2021), the OWASP Application Security Verification Standard (OWASP Foundation, n.d.-a) and the OWASP Cheat Sheet Series (OWASP Foundation, n.d.-b) will define what secure behaviour looks like and what to test. Product analysis can

compare intentionally vulnerable practice applications such as OWASP Juice Shop and DVWA to see how they structure weaknesses and test cases. A discussion with the client can check the scope, the safety rules for the EDU Cloud and the tools that are allowed. Small tool trials with Nmap (Nmap Project, n.d.), Wireshark (Wireshark Foundation, n.d.) and tcpdump on a test machine will confirm that the planned evidence can actually be collected. These are planned activities, not completed research results.

4.2 Scope decisions

One user role, plain text notes and a single deployment per environment are the baseline. Sharing, uploads and extra roles are deferred. Fastify is the preferred backend framework because its built-in schema validation supports the input validation goal. The security findings do not depend on the choice, so Express remains an acceptable fallback.

The following weaknesses are planned for the unsecured version and are mapped to the OWASP Top 10 (OWASP Foundation, 2021). Whether each can be reproduced is not yet confirmed.

Planned weakness	How it will be tested	Planned fix (secured version)	OWASP Top 10 area
Weak password storage	Inspect the stored values in the database.	Argon2 or bcrypt hashing.	Cryptographic Failures
SQL injection	Send malicious input to login and note fields.	Parameterized queries.	Injection
Cross-site scripting	Store script payloads in a note and view it.	Output encoding, validation and a Content-Security-Policy.	Injection
Broken authorization	Change the note id in a request as another user.	Ownership check on every note query.	Broken Access Control
No HTTPS	Capture login traffic with Wireshark.	TLS in Nginx and redirect from HTTP.	Cryptographic Failures
Unnecessary open ports	Nmap scan of the host.	UFW rules and no published database port.	Security Misconfiguration
No rate limiting	Repeated login attempts with curl.	Rate limiting on login.	Identification and Authentication Failures
Limited logging	Perform attacks, then review the logs.	Structured security logging.	Security Logging and Monitoring Failures
Weak session handling	Inspect cookies and tokens in browser tools.	HttpOnly, Secure and SameSite cookies with expiry.	Identification and Authentication Failures
Verbose error messages	Trigger errors on purpose.	Generic errors for users, details only in server logs.	Security Misconfiguration

The number of notes per user and the exact password policy are not yet specified. Before testing, the chosen values will be recorded and used consistently in both versions. The acceptance checks in section 6 form the basis for a separate test record with actual results.

5 Requirement priorities

Must have requirements define a complete and testable project. Should have requirements improve the hardened version once the core works. Could have items depend on remaining time. Won’t have items are deliberately excluded from this version.

Priority	Requirements
Must have	REQ-01 to REQ-14: the SecureNotes functions, the isolated unsecured version, the database, the required tests, the fixes, the retest and the VPS deployment (section 6).
Should have	REQ-15 Rate limiting on login. REQ-16 Security logging of failed logins and denied access. REQ-17 Security headers (Content-Security-Policy, HSTS). REQ-18 Cookie flags HttpOnly, Secure and SameSite. REQ-19 SSH hardening on the VPS (key authentication, no password login).
Could have	Fail2ban on the VPS; containers running as a non-root user; dependency scanning with npm audit; a script that repeats the test cases automatically.
Won’t have	Note sharing; file uploads; password reset by email; multi-factor authentication; multi-server or high-availability setups; any test against systems that do not belong to the client.

6 Must have requirements

Each acceptance check describes how to verify the finished project. The checks are planned and do not imply a passing result.

ID	Requirement	Acceptance check
REQ-01	A user can create an account.	Register with a new username and password. The account exists and login works. A duplicate username is rejected.
REQ-02	A user can log in and log out.	Log in with correct credentials and reach the notes page. Wrong credentials are rejected. After logout, protected pages are not accessible.
REQ-03	A logged-in user can create a note.	Create a note and confirm it is stored and listed.
REQ-04	A user can view their own notes.	Log in and confirm only that user’s notes are shown.
REQ-05	A user can edit their own notes.	Edit a note and confirm the change persists after reload.
REQ-06	A user can delete their own notes.	Delete a note and confirm it no longer appears or can be fetched.
REQ-07	The unsecured version runs only in an isolated environment.	Start the unsecured version in the EDU Cloud, VM or isolated Docker network. Confirm from an outside network that it is not reachable.
REQ-08	Application data is stored in PostgreSQL.	Inspect the database and confirm users and notes are stored there.
REQ-09	Ports are scanned with Nmap on both versions.	Save the Nmap output for the unsecured and secured versions. The secured version shows only ports 22, 80 and 443.
REQ-10	HTTP and HTTPS traffic are captured and compared.	Capture a login with Wireshark or tcpdump on each version. Credentials are readable on the unsecured version and not readable on the secured version.
REQ-11	At least six vulnerability types are tested and documented on the unsecured version.	Each finding has a description, reproduction steps, evidence and an explanation of why it is a security problem.
REQ-12	The secured version fixes every documented finding.	Each finding has a documented fix in the code or the configuration.

ID	Requirement	Acceptance check
REQ-13	The secured version is retested with the same test cases.	Each test from REQ-11 is repeated and its result recorded next to the unsecured version result.
REQ-14	The secured version is deployed on the VPS behind Nginx with HTTPS.	Open the site over HTTPS. HTTP redirects to HTTPS. PostgreSQL and the API port are not reachable from outside.

7 Risks and validation priorities

Risk	Effect	Mitigation and validation
Scope creep into a full-featured app	Less time for testing and retesting.	Keep to the Must have requirements. Defer extras.
Too little time for testing after the build	Weak evidence for the findings.	Time-box development, write test cases early and record evidence while testing.

The validation priorities, in order, are:

1. Build a minimal prototype of the unsecured version and confirm each planned weakness can be reproduced.
2. Write the test cases before attacking, so the before and after results are comparable.
3. Test the unsecured version, fix the findings in the secured version and retest with the same cases.
4. Deploy the secured version to the VPS and verify the exposed ports and HTTPS from outside.

References

Docker. (n.d.). *Docker docs*. https://docs.docker.com/

Express. (n.d.). *Express - Node.js web application framework*. https://expressjs.com/

Fastify. (n.d.). *Fastify documentation*. https://fastify.dev/docs/latest/

Nmap Project. (n.d.). *Nmap reference guide*. https://nmap.org/book/man.html

OWASP Foundation. (n.d.-a). *OWASP Application Security Verification Standard (ASVS)*. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (n.d.-b). *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). *Technical guide to information security testing and assessment* (NIST Special Publication 800-115). National Institute of Standards and Technology. https://csrc.nist.gov/pubs/sp/800/115/final

Ubuntu. (n.d.). *UFW*. https://help.ubuntu.com/community/UFW

Wireshark Foundation. (n.d.). *Wireshark user's guide*. https://www.wireshark.org/docs/